Subtopic: Parallelism

---

1. Introduction to Parallelism

- **Definition:** Parallelism refers to the ability of a computer system to perform multiple operations **simultaneously** to improve performance.
- Goal: Reduce execution time and increase system throughput.
- Modern computing heavily relies on parallelism due to the **limitations of clock speed scaling** (power consumption, heat, etc.).

---

2. Types of Parallelism

A. Instruction-Level Parallelism (ILP)

- Multiple instructions are executed in parallel within a single processor.
- Achieved through:
    - **Pipelining:** Overlaps different stages of instruction execution.
    - **Superscalar Execution:** Multiple execution units handle more than one instruction per cycle.
    - **Out-of-Order Execution:** Instructions are executed as soon as operands are ready, not strictly in program order.

B. Data-Level Parallelism (DLP)

- Same operation performed on multiple data items simultaneously.

- Used in **vector processors** and **SIMD (Single Instruction, Multiple Data)**.
- Example: Adding two large arrays in one instruction cycle.

C. Thread-Level Parallelism (TLP)

- Multiple threads (lightweight processes) execute in parallel.
- Implemented in **multi-core processors** where each core handles one or more threads.
- Example: Web browser running multiple tabs simultaneously.

D. Process-Level Parallelism (PLP)

- Entire processes run concurrently.
- Achieved through **multiprocessing systems** (SMP, distributed systems, clusters).
- Example: A server handling thousands of client requests concurrently.

---

3. Flynn's Taxonomy of Parallelism

Michael Flynn (1966) classified computer systems into four categories:

1. **SISD (Single Instruction, Single Data):** Traditional sequential computers.
2. **SIMD (Single Instruction, Multiple Data):** One instruction operates on multiple data (vector processors, GPUs).

3. **MISD (Multiple Instruction, Single Data):** Rare; multiple instructions on the same data stream.

4. **MIMD (Multiple Instruction, Multiple Data):** Multiple processors execute different instructions on different data (multi-core CPUs, clusters).

---

4. Hardware Support for Parallelism

- **Multiple Cores:** Processors with 2, 4, 8, 16+ cores.
- **Vector Units / SIMD Extensions:** SSE, AVX (Intel), NEON (ARM).
- **GPUs (Graphics Processing Units):** Thousands of simple cores optimized for DLP.
- **Caches:** Shared and private caches reduce memory bottlenecks in parallel execution.

---

5. Challenges in Parallelism

1. **Data Dependencies:**
    - RAW (Read After Write), WAR (Write After Read), WAW (Write After Write) dependencies limit ILP.
2. **Synchronization Issues:**
    - Multiple threads/processes must coordinate (e.g., critical sections, locks).
3. **Memory Bottleneck:**
    - Multiple cores competing for main memory slows down execution.

4. **Amdahl's Law:**
    - Speedup is limited by the **sequential portion** of the program.
    - Example: If 10% of a program is sequential, maximum speedup = 10x, no matter how many processors are added.

---

6. Applications of Parallelism

- **Scientific Computing:** Weather simulations, DNA sequencing.
- **Graphics Processing:** Image rendering, 3D games, AI/ML training.
- **Databases:** Parallel queries and transaction processing.
- **Web Servers and Cloud Systems:** Handling concurrent requests.

✅ **Summary**

- Parallelism improves performance by allowing **multiple instructions, data items, threads, or processes** to execute simultaneously.
- Classified into **ILP, DLP, TLP, PLP**.
- **Flynn's taxonomy** provides a structured classification (SISD, SIMD, MISD, MIMD).
- Hardware (multi-core CPUs, GPUs, vector units) plays a major role.
- Challenges include **data dependencies, synchronization, memory bottlenecks, and limits from Amdahl's Law**.